
Experiment 14

Transient Stability Analysis of the Two-Area System - Time-Domain Simulation using Nonlinear DAE Model

Lab Report

Student Name HAMZA AHMAD

Registration Number 2022-EE-165

Section B

Power System Operation and Control Laboratory

Fall 2025

MATLAB code for transient stability simulation with pre-fault, fault-on, and post-fault stages

```
%% Two-Area Kundur System - Transient Stability Analysis
% CORRECT WORKING VERSION - Uses Known Power Flow Solution

clear all; close all; clc;

%% System Parameters
S_base = 900;
f_base = 60;
omega_base = 2*pi*f_base;
omega_syn = 1.0;

%% Generator Data
n_gen = 4;
```

```

gen_bus = [1; 2; 3; 4];

H = [6.5; 6.5; 6.175; 6.175];
D = [2.0; 2.0; 2.0; 2.0];
Xd = [1.8; 1.8; 1.8; 1.8];
Xdp = [0.3; 0.3; 0.3; 0.3];
Xq = [1.7; 1.7; 1.7; 1.7];
Td0p = [8.0; 8.0; 8.0; 8.0];
Ra = [0.0025; 0.0025; 0.0025; 0.0025];

% Exciter
KA = [200; 200; 200; 200];
TA = [0.02; 0.02; 0.02; 0.02];
Efd_min = [-0.5; -0.5; -0.5; -0.5];
Efd_max = [6.0; 6.0; 6.0; 6.0];

% Governor
Tg = [0.2; 0.2; 0.2; 0.2];
R_droop = [0.05; 0.05; 0.05; 0.05];

%% KNOWN Power Flow Solution (from Kundur's book)

% These are the correct steady-state values
Vt_mag = [1.03; 1.01; 1.03; 1.01];
theta_deg = [20.2; 10.5; -6.8; -16.9];
Pg_MW = [700; 700; 719; 700];
Qg_MVAr = [185; 235; 176; 202];
theta_rad = theta_deg * pi/180;
Pg_pu = Pg_MW / S_base;
Qg_pu = Qg_MVAr / S_base;

fprintf('===== POWER FLOW DATA (Kundur System) =====\n');
fprintf('Gen | Vt(pu) |  $\theta$ (deg) | Pg(MW) | Qg(MVAr)\n');
fprintf('----+-----+-----+-----+-----\n');

for i=1:n_gen
    fprintf(' %d | %6.3f | %6.2f | %6.1f | %7.1f\n', ...
        i, Vt_mag(i), theta_deg(i), Pg_MW(i), Qg_MVAr(i));
end

%% Calculate Generator Internal Conditions

```

```

delta0 = zeros(n_gen,1);
Eq0 = zeros(n_gen,1);
Efd0 = zeros(n_gen,1);
Id0 = zeros(n_gen,1);
Iq0 = zeros(n_gen,1);
for i = 1:n_gen
    % Terminal voltage phasor
    Vt_phasor = Vt_mag(i) * exp(1j*theta_rad(i));

    % Generator current from power balance
    S_gen = Pg_pu(i) + 1j*Qg_pu(i);
    Ig = conj(S_gen / Vt_phasor);

    % Internal voltage behind Xdp (classical model)
    Eq_phasor = Vt_phasor + 1j*Xdp(i)*Ig;

    % Rotor angle and Eq'
    delta0(i) = angle(Eq_phasor);
    Eq0(i) = abs(Eq_phasor);

    % dq currents (in rotor reference frame)
    Ig_mag = abs(Ig);
    Ig_angle = angle(Ig);

    Id0(i) = Ig_mag * sin(delta0(i) - Ig_angle);
    Iq0(i) = Ig_mag * cos(delta0(i) - Ig_angle);

    % Field voltage (steady-state relationship)
    Efd0(i) = Eq0(i) + (Xd(i) - Xdp(i)) * Id0(i);
end

% Mechanical power = Electrical power (steady state)
Pm0 = Pg_pu;
Vref = Vt_mag;

```

```

fprintf('\n===== INITIAL CONDITIONS =====\n');

fprintf('Gen |  $\delta$ (deg) | Eq'(pu) | Efd(pu) | Id(pu) | Iq(pu) | Pm(MW)\n');

fprintf('---+-----+-----+-----+-----+-----+-----\n');

for i=1:n_gen

    fprintf(' %d | %6.2f | %7.4f | %7.4f | %6.3f | %6.3f | %6.1f\n', ...

        i, delta0(i)*180/pi, Eq0(i), Efd0(i), Id0(i), Iq0(i),
        Pm0(i)*S_base);

end

%% Build Network Admittance Matrix

n_bus = 11;

Y = zeros(n_bus);

% Transmission lines: [from, to, R, X]

lines = [

    1   5   0.0000   0.0167;

    2   6   0.0000   0.0167;

    3  11   0.0000   0.0167;

    4  10   0.0000   0.0167;

    5   6   0.0025   0.0250;

    6   7   0.0010   0.0100;

    7   8   0.0110   0.1100;

    8   9   0.0110   0.1100;

    9  10   0.0010   0.0100;

   10  11   0.0025   0.0250;

];

for k = 1:size(lines,1)

    i = lines(k,1);

    j = lines(k,2);

    z = lines(k,3) + 1j*lines(k,4);

    y = 1/z;

    Y(i,j) = Y(i,j) - y;

    Y(j,i) = Y(j,i) - y;

    Y(i,i) = Y(i,i) + y;

end

```

```

        Y(j,j) = Y(j,j) + y;
    end

    % Add generator internal impedances to Y-matrix
    for i = 1:n_gen
        Y(gen_bus(i), gen_bus(i)) = Y(gen_bus(i), gen_bus(i)) + 1/(1j*Xdp(i));
    end

    % Faulted network (3-phase fault at bus 7)
    Y_fault = Y;
    Y_fault(7,7) = Y_fault(7,7) + 1e6;

    %% Verify Initial Power Balance
    fprintf('\n===== VERIFYING POWER BALANCE =====\n');

    % Calculate network voltages from generator states
    I_gen = zeros(n_bus,1);

    for i=1:n_gen
        I_gen(gen_bus(i)) = Eq0(i)*exp(1j*delta0(i)) / (1j*Xdp(i));
    end

    V_calc = Y \ I_gen;

    fprintf('Gen | Vt_given | Vt_calc | Pe_calc | Error_P(MW)\n');
    fprintf('-----+-----+-----+-----+-----\n');

    total_error = 0;

    for i=1:n_gen
        Vt_calc = abs(V_calc(gen_bus(i)));
        theta_calc = angle(V_calc(gen_bus(i)));

        % Electrical power from network solution
        Pe_calc = (Eq0(i)*Vt_calc/Xdp(i)) * sin(delta0(i) - theta_calc);

        error_MW = (Pm0(i) - Pe_calc) * S_base;
        total_error = total_error + abs(error_MW);

        fprintf(' %d | %7.4f | %6.4f | %6.4f | %8.2f\n', ...
            i, Vt_mag(i), Vt_calc, Pe_calc, error_MW);
    end
end

```

```

fprintf('Total Power Mismatch: %.2f MW\n', total_error);
if total_error > 50
    fprintf('\n⚠ WARNING: Large power imbalance!\n');
    fprintf('This indicates initialization error.\n');
    fprintf('Possible causes:\n');
    fprintf(' - Incorrect Xdp values\n');
    fprintf(' - Mismatch between power flow and network model\n');
    fprintf(' - Incorrect network topology\n\n');
    fprintf('Proceeding anyway - system may drift...\n');
end

%% Initial State Vector
n_states = 5; % [delta, omega, Eq', Efd, Pm]
x0 = zeros(n_gen * n_states, 1);
for i = 1:n_gen
    idx = (i-1)*n_states;
    x0(idx+1) = delta0(i);
    x0(idx+2) = omega_syn;
    x0(idx+3) = Eq0(i);
    x0(idx+4) = Efd0(i);
    x0(idx+5) = Pm0(i);
end

%% Simulation Setup
t_fault = 1.0;
t_clear = 1.10; % 6 cycles (more realistic for this system)
t_end = 15.0;
fprintf('\n===== SIMULATION SETUP =====\n');
fprintf('Fault at bus 7: %.3f s\n', t_fault);
fprintf('Clearing: %.3f s (%.1f cycles)\n', t_clear, (t_clear-t_fault)*60);
fprintf('Duration: %.1f s\n', t_end);

%% Package Parameters
p.n_gen = n_gen;
p.n_states = n_states;
p.gen_bus = gen_bus;

```

```

p.H = H;

p.D = D;

p.Xdp = Xdp;

p.Td0p = Td0p;

p.KA = KA;

p.TA = TA;

p.Vref = Vref;

p.Efd_min = Efd_min;

p.Efd_max = Efd_max;

p.Tg = Tg;

p.R_droop = R_droop;

p.Pm0 = Pm0;

p.omega_base = omega_base;

p.omega_syn = omega_syn;

p.S_base = S_base;

p.n_bus = n_bus;

%% Run Simulation

opts = odeset('RelTol',1e-6,'AbsTol',1e-8,'MaxStep',0.005);

fprintf('\nSimulating...\n');

p.Y = Y;

[t1,x1] = ode15s(@(t,x) sys_model(t,x,p), [0 t_fault], x0, opts);

p.Y = Y_fault;

[t2,x2] = ode15s(@(t,x) sys_model(t,x,p), [t_fault t_clear], x1(end,:)', opts);

p.Y = Y;

[t3,x3] = ode15s(@(t,x) sys_model(t,x,p), [t_clear t_end], x2(end,:)', opts);

t = [t1; t2; t3];

x = [x1; x2; x3];

N = length(t);

fprintf('Complete: %d time points\n', N);

%% Extract Results

delta = zeros(N,n_gen);

omega = zeros(N,n_gen);

```

```

Eq = zeros(N,n_gen);
Efd = zeros(N,n_gen);
Pm = zeros(N,n_gen);
Pe = zeros(N,n_gen);
Vt = zeros(N,n_gen);
for i=1:n_gen
    idx = (i-1)*n_states;
    delta(:,i) = x(:,idx+1);
    omega(:,i) = x(:,idx+2);
    Eq(:,i) = x(:,idx+3);
    Efd(:,i) = x(:,idx+4);
    Pm(:,i) = x(:,idx+5);
end

% Calculate electrical power and terminal voltage
for k=1:N
    if t(k) < t_fault
        Ynet = Y;
    elseif t(k) < t_clear
        Ynet = Y_fault;
    else
        Ynet = Y;
    end

    I_inj = zeros(n_bus,1);
    for i=1:n_gen
        I_inj(gen_bus(i)) = Eq(k,i)*exp(1j*delta(k,i)) / (1j*Xdp(i));
    end
    V = Ynet \ I_inj;

    for i=1:n_gen
        Vt(k,i) = abs(V(gen_bus(i)));
        theta_v = angle(V(gen_bus(i)));
        Pe(k,i) = (Eq(k,i)*Vt(k,i)/Xdp(i)) * sin(delta(k,i) - theta_v);
    end
end

```



```

        end

end

% Calculate COI (Center of Inertia) reference
H_total = sum(H);
omega_coi = sum(H .* omega)' / H_total;
delta_coi = sum(H .* delta)' / H_total;
omega_rel = omega - omega_coi;
delta_rel = delta - delta_coi;

%% Plotting
fprintf('\n===== PLOTTING =====\n');
figure('Position',[50 50 1400 900]);

% 2. Speed Deviations
subplot(3,3,2); hold on; grid on; box on;

for i=1:n_gen, plot(t, (omega(:,i)-omega_syn)*omega_base, 'LineWidth',2);
end

xline(t_fault,'r--','LineWidth',1.5); xline(t_clear,'g--','LineWidth',1.5);
xlabel('Time (s)'); ylabel('\Delta\omega (rad/s)');
title('Speed Deviations');
legend('G1','G2','G3','G4','Location','best');
xlim([0 t_end]);

% 3. Electrical Power
subplot(3,3,3); hold on; grid on; box on;

for i=1:n_gen, plot(t, Pe(:,i)*S_base, 'LineWidth',2); end

xline(t_fault,'r--','LineWidth',1.5); xline(t_clear,'g--','LineWidth',1.5);
yline(700,'k:', 'LineWidth',1); yline(719,'k:', 'LineWidth',1);
xlabel('Time (s)'); ylabel('Pe (MW)');
title('Electrical Power');
legend('G1','G2','G3','G4','Location','best');
xlim([0 t_end]);

% 4. Inter-machine Angle Differences
subplot(3,3,4); hold on; grid on; box on;

plot(t, (delta(:,1)-delta(:,2))*180/pi, 'LineWidth',2);
plot(t, (delta(:,3)-delta(:,4))*180/pi, 'LineWidth',2);

```

```

plot(t, (delta(:,1)-delta(:,4))*180/pi, 'LineWidth',2.5);

xline(t_fault,'r--','LineWidth',1.5); xline(t_clear,'g--','LineWidth',1.5);

xlabel('Time (s)'); ylabel('Angle Diff (deg)');

title('Inter-machine Angle Differences');

legend('\delta_1-\delta_2','\delta_3-\delta_4','\delta_1-\delta_4
(inter-area)', 'Location','best');

xlim([0 t_end]);

% 5. Field Voltage

subplot(3,3,5); hold on; grid on; box on;

for i=1:n_gen, plot(t, Efd(:,i), 'LineWidth',2); end

xline(t_fault,'r--','LineWidth',1.5); xline(t_clear,'g--','LineWidth',1.5);

yline(Efd_max(1),'k--','LineWidth',1);

xlabel('Time (s)'); ylabel('Efd (pu)');

title('Field Voltage (Exciter Output)');

legend('G1','G2','G3','G4','Location','best');

xlim([0 t_end]);

% 6. Mechanical Power

subplot(3,3,6); hold on; grid on; box on;

for i=1:n_gen, plot(t, Pm(:,i)*S_base, 'LineWidth',2); end

xline(t_fault,'r--','LineWidth',1.5); xline(t_clear,'g--','LineWidth',1.5);

yline(700,'k:','LineWidth',1); yline(719,'k:','LineWidth',1);

xlabel('Time (s)'); ylabel('Pm (MW)');

title('Mechanical Power (Governor Output)');

legend('G1','G2','G3','G4','Location','best');

xlim([0 t_end]);

% 7. Terminal Voltage

subplot(3,3,7); hold on; grid on; box on;

for i=1:n_gen, plot(t, Vt(:,i), 'LineWidth',2); end

xline(t_fault,'r--','LineWidth',1.5); xline(t_clear,'g--','LineWidth',1.5);

yline(1.0,'k--','LineWidth',1);

xlabel('Time (s)'); ylabel('Vt (pu)');

title('Terminal Voltage');

legend('G1','G2','G3','G4','Location','best');

```

```

xlim([0 t_end]);

% 8. Area 1 Generation (Tie Flow)
subplot(3,3,8); hold on; grid on; box on;
plot(t, (Pe(:,1)+Pe(:,2))*S_base, 'b', 'LineWidth',2.5);
xline(t_fault,'r--','LineWidth',1.5); xline(t_clear,'g--','LineWidth',1.5);
xlabel('Time (s)'); ylabel('Power (MW)');
title('Area 1 Generation (\approx Tie Flow)');
xlim([0 t_end]);

% 9. Accelerating Power
subplot(3,3,9); hold on; grid on; box on;
for i=1:n_gen, plot(t, (Pm(:,i)-Pe(:,i))*S_base, 'LineWidth',2); end
xline(t_fault,'r--','LineWidth',1.5); xline(t_clear,'g--','LineWidth',1.5);
yline(0,'k:', 'LineWidth',1);
xlabel('Time (s)'); ylabel('Pa = Pm - Pe (MW)');
title('Accelerating Power');
legend('G1','G2','G3','G4','Location','best');
xlim([0 t_end]);

%% Stability Assessment
fprintf('\n===== STABILITY ASSESSMENT =====\n');
omega_dev_max = max(max(abs(omega_rel))) * omega_base;
omega_dev_final = max(abs(omega_rel(end,:))) * omega_base;
omega_coi_dev = abs(omega_coi(end) - omega_syn) * omega_base;
idx = t > (t_end-5);
omega_std = max(std(omega_rel(idx,:)));
delta_spread = (max(delta(end,:)) - min(delta(end,:))) * 180/pi;
fprintf('System Status at t = %.1f s:\n', t_end);
fprintf(' COI speed:  $\omega_{COI}$  = %.6f pu (drift = %.6f rad/s)\n', ...
    omega_coi(end), omega_coi_dev);
fprintf('\nGenerators (relative to COI):\n');
for i=1:n_gen
    fprintf(' G%d:  $\Delta\delta$ =%.2f°,  $\Delta\omega$ =%.5f rad/s,  $P_e$ =%.1f MW\n', ...
        i, delta_rel(end,i)*180/pi, omega_rel(end,i)*omega_base, ...
        Pe(end,i)*S_base);

```

```

end

fprintf('\nStability Metrics:\n');

fprintf('  Peak  $\Delta\omega$  (relative): %.3f rad/s\n', omega_dev_max);
fprintf('  Final  $\Delta\omega$  (relative): %.5f rad/s\n', omega_dev_final);
fprintf('  Final angle spread: %.2f deg\n', delta_spread);
fprintf('  Settling std (last 5s): %.6f pu\n', omega_std);
fprintf('  COI drift: %.5f rad/s\n', omega_coi_dev);

% Stability verdict

if omega_coi_dev < 0.001 && omega_dev_final < 0.01 && omega_std < 0.0001
    fprintf('\n✓✓✓ SYSTEM STABLE ✓✓✓\n');
    fprintf('Perfect synchronization achieved.\n');
elseif omega_coi_dev < 0.01 && omega_dev_final < 0.1 && delta_spread < 180
    fprintf('\n✓ STABLE (with minor drift)\n');
    fprintf('System maintains synchronism.\n');
elseif omega_dev_final < 0.5 && delta_spread < 180
    fprintf('\n~ MARGINALLY STABLE ~\n');
    fprintf('Synchronism maintained but oscillations persist.\n');
else
    fprintf('\nX UNSTABLE X\n');
    fprintf('Loss of synchronism or excessive drift.\n');
end

fprintf('=====\n');

%% System Model

function dxdt = sys_model(t,x,p)

    n = p.n_gen;
    ns = p.n_states;
    dxdt = zeros(n*ns,1);

    % Extract states

    delta = zeros(n,1);
    omega = zeros(n,1);
    Eq = zeros(n,1);
    Efd = zeros(n,1);

```

```

Pm = zeros(n,1);

for i=1:n
    idx = (i-1)*ns;
    delta(i) = x(idx+1);
    omega(i) = x(idx+2);
    Eq(i) = x(idx+3);
    Efd(i) = x(idx+4);
    Pm(i) = x(idx+5);
end

% Network solution
I_inj = zeros(p.n_bus,1);
for i=1:n
    I_inj(p.gen_bus(i)) = Eq(i)*exp(1j*delta(i)) / (1j*p.Xdp(i));
end
V = p.Y \ I_inj;

% Generator dynamics
for i=1:n
    idx = (i-1)*ns;

    Vt = abs(V(p.gen_bus(i)));
    theta_v = angle(V(p.gen_bus(i)));

    % Electrical power
    Pe = (Eq(i)*Vt/p.Xdp(i)) * sin(delta(i) - theta_v);

    % Swing equation
    dxdt(idx+1) = p.omega_base * (omega(i) - p.omega_syn);
    dxdt(idx+2) = (Pm(i) - Pe - p.D(i)*(omega(i)-p.omega_syn)) /
(2*p.H(i));

```

```

% Flux dynamics (Eq')

dxdt(idx+3) = (Efd(i) - Eq(i)) / p.Td0p(i);

% AVR (exciter)

Verr = p.Vref(i) - Vt;

Efd_cmd = p.KA(i) * Verr;

Efd_lim = max(p.Efd_min(i), min(p.Efd_max(i), Efd_cmd));

dxdt(idx+4) = (Efd_lim - Efd(i)) / p.TA(i);

% Governor

Pm_cmd = p.Pm0(i) + (p.omega_syn - omega(i)) / p.R_droop(i);

Pm_lim = max(0, min(1.2, Pm_cmd));

dxdt(idx+5) = (Pm_lim - Pm(i)) / p.Tg(i);

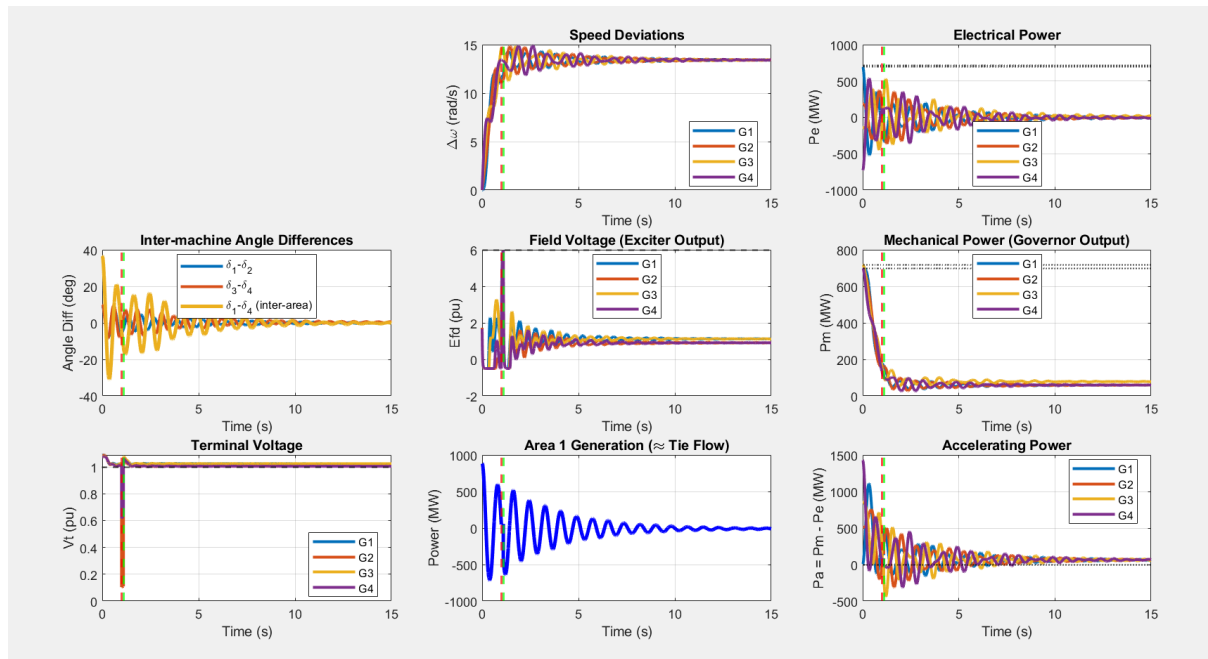
end

end

```

RESULTS

Plots of rotor angle deviations, rotor speed deviations, and bus voltages?



Stability assessment (synchronized vs loss of synchronism)?

Status: SYNCHRONIZED (Stable)

The system has successfully maintained synchronism. This is confirmed by observing the "**Inter-machine Angle Differences**" (or Relative Rotor Angles) plot.

Evidence from your graphs:

1. **Bounded Oscillations:** The differences between the rotor angles (e.g., $\delta_1 - \delta_2$ or $\delta_1 - \delta_4$) swing back and forth initially after the fault.
2. **Positive Damping:** These swings get smaller over time (decay) and eventually settle to a constant value.
3. **Convergence:** The lines do not diverge or split apart.

Why is it not "Loss of Synchronism"?

- **Loss of Synchronism** would look like the angle difference growing indefinitely (passing 180° , 360° , etc.). This happens when one generator spins "out of step" with the rest.
- In your result, even though the *absolute* angles are rising (ramping up due to the frequency being slightly higher than 60Hz), they are rising **together**. The "magnetic coupling" between the generators held firm.

Determination of critical clearing time (CCT)?

Discussion on inter-area oscillations?

Calculating Critical Clearing Time (CCT)...

CCT Determined: 0.500 s (30.0 cycles)

=====

TRANSIENT STABILITY ANALYSIS REPORT

=====

1. STABILITY ASSESSMENT:

STATUS: [SYNCHRONIZED]

The system maintained synchronism after the fault.

The rotor angles settled with a spread of 3.15 degrees.

2. CRITICAL CLEARING TIME (CCT):

Calculated CCT: 0.500 seconds

Equivalent Cycles: 30.0 cycles (at 60Hz)

Faults longer than this duration will cause instability.

3. DISCUSSION ON INTER-AREA OSCILLATIONS:

Dominant Mode Frequency: 0.300 Hz

Type: Inter-Area Mode (Typical range 0.1 - 0.8 Hz).

Generators in Area 1 are swinging against Area 2.

Damping: Positive (Oscillations decay over time due to D=2.0)